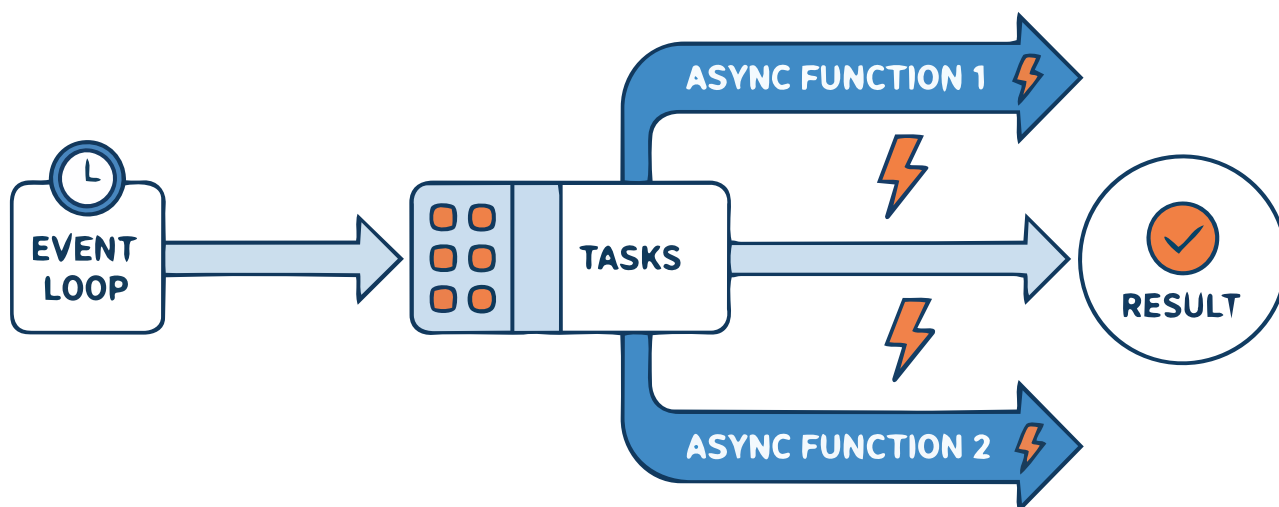


The Basics of Asynchronous Programming in Python

Asynchronous programming allows code to handle multiple operations concurrently without blocking the execution of the program. It's particularly useful for I/O-bound tasks (network requests, file operations, database queries) where waiting for external resources would otherwise slow down the program.



A synchronous programming is used to implement non-blocking operations for I/O bound tasks using the concept of ‘cooperative multitasking’. Unlike pre-emptive multitasking, where CPU is allocated to threads based on the scheduling algorithm, priority, idle time, etc, event loops control execution without explicitly creating user threads in asynchronous programming.

Multitasking (Threads/Processes): This is best for CPU-bound tasks like computation, encryption, compression, AI inference. The OS pre-empts running threads and switches between them using the CPU scheduler.

Asynchronous programming (Async/Await): This is best for I/O-bound tasks where the program spends most of its time waiting. Examples are:

- Thousands of HTTP requests
- Waiting for database queries
- Calling remote APIs
- Reading/writing slow disks

Async ensures the CPU does not sit idle during these waits.

In synchronous code, operations run one after another. If one task waits for I/O (like network or disk), the entire program pauses. Asynchronous programming allows Python to run multiple tasks concurrently on one thread, using the event loop.

When one task waits, Python immediately switches to another, improving responsiveness and throughput.

Key concepts of asynchronous programming are:

- **Event loop:** This is the Python ‘scheduler’ that runs asynchronous tasks. It decides which tasks should run and when to pause/resume them.
- **Coroutine:** This refers to cooperative multitasking — a function defined using `async def`. It waits on the `await` keyword and explicitly surrenders control to the CPU calling `await`.
- **Await:** This is used to pause the current coroutine in